Methodology_Results

Methodology

Data Summary

Table 1: Variable Descriptions

variable	class	description
crl.tot	double	Total length of uninterrupted sequences of capitals
dollar	double	Occurrences of the dollar sign, as percent of total number of characters
bang	double	Occurrences of '!', as percent of total number of characters
money	double	Occurrences of 'money', as percent of total number of characters
n000	double	Occurrences of the string '000', as percent of total number of words
make	double	Occurrences of 'make', as a percent of total number of words
yesno	character	Outcome variable, a factor with levels 'n' not spam, 'y' spam

The dataset includes a sample of 4,601 observations of emails which are classified as spam (y) or non-spam (n). It also includes other attributes of each mail such as frequencies of different words, occurrences of different symbols, and lengths of capital letters, which may help distinguish spam from non-spam emails.

The dataset does not contain any missing value, but we we noticed that some rows were completely identical when initially inspecting the dataset. This suggests that the data contains emails with possibly the exact same content appearing multiple times. This could be due to spam emails being sent to multiple recipients or non-spam emails, such as announcements, being distributed widely. When handling these observations, we should not treat them distinctively because they may essentially represent the same email. Keeping these duplicates could potentially impact future model training by introducing redundancy, leading to biases in learning patterns.

When initially examining the data, we observed an imbalance in the proportions of spam and non-spam emails, with non-spam emails being the majority in the original dataset. After extracting the duplicate data, we found that a large proportion of the duplicates were non-spam emails. After removing these duplicates, we noticed a

significant reduction in the number of non-spam emails (as presented in Figure 1), resulting in a more balanced dataset between spam and non-spam emails. We consider this change to be a positive outcome, as a more balanced dataset can help improve generalization and avoid model bias toward the majority class. We believe this will better support the training of machine learning models, leading to more accurate and reliable predictions.

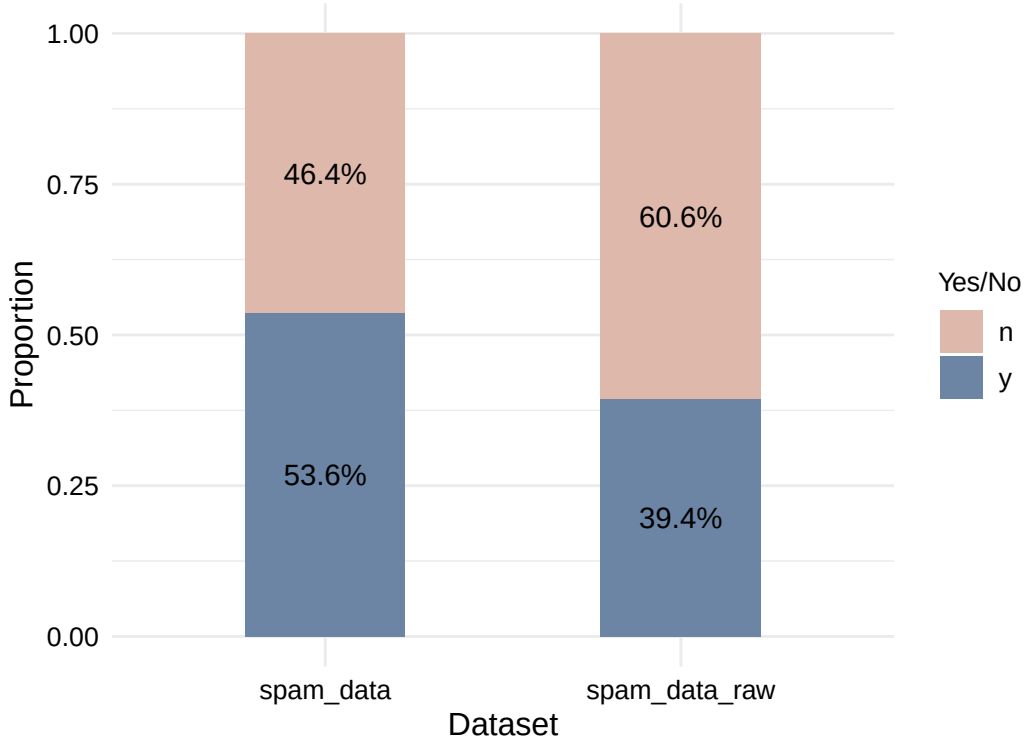

Figure 1: Proportion of Yes/No Before and After Dropping Duplicates

Table 2 indicate that spam emails generally show more outliers than non-spam emails in attributes such as 'crl.tot', 'money', and 'n000', while non-spam emails tend to show more outliers in attributes like 'dollar', 'bang', and 'make'. However, when we look at the boxplots (Please see Figure 4 in appendix) to assess the level of extreme outliers, we observe that spam emails tend to have more extreme outliers in most attributes except for the occurrences of "!". These findings suggest that spam emails often use more extreme patterns, particularly with attention-grabbing elements like capital letters, dollar signs, and certain word occurrences, while non-spam emails might display more variability in certain other features like punctuation and specific words.

Table 2: Outliers of Each Attributes in Spam VS Non-Spam Emails

	Spam_Outliers	Non_Spam_Outliers
crl.tot	166	128
dollar	111	286
bang	90	135
money	124	54
n000	148	76
make	200	266

Data Pre-processing

Since our data contains many outliers and the scale of each feature varies significantly, we need to conduct normalization first before training the model. Many machine learning algorithms perform better when the features are on a similar scale. Without normalization, feature with larger scale could dominate in the model learning process, leading to biased results.

In this case, min-max scaling is because it is simple and straightforward while keeping the outlier patterns for spam detection. It also ensures that the model treats all features equally and is less sensitive to extreme outliers.

Furthermore, as the original dataset distinguish spam and non-spam emails as y's and n's. They are transformed to 1s and 0s in this process.

Feature Selection

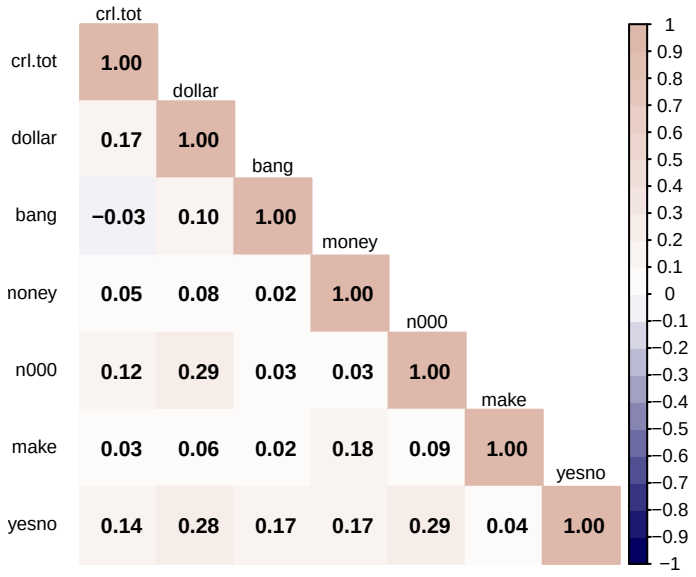

Figure 2: Correlation Heatmap of Numeric Features

No predictors have strong correlations (>0.7) with each other, so multicollinearity isn't a major concern among our dataset. However, we can observe the highest correlation is 0.29 between "dollar" and "n000". This correlation is mild and we will keep them for now and consider the removal of one of them in later model performance analysis.

As for feature selection for target variable prediction, 'n000' and 'dollar' are the most correlated with 'yesno', followed by 'money', 'bang', and 'crl.tot'. 'Make' has the lowest correlation of 0.04, it will be removed from the independent variables.

Model Selection

For this project, as we are conducting binary classification, we will begin with logistic regression and decision trees because they are simple, easy to interpret, and effective. However, due to the large number of outliers among each feature, other machine learning models (e.g. support vector machines (SVM)) will also be considered if the initial model selections do not provide satisfactory performance.

Results

Model Performance Evaluation

The dataset is splitted into two subsets, with 70% allocated for model training and 30% reserved for testing. The confusion matrices for the logistic regression and decision tree models are presented in Table 3 and Table 4.

(Please see detailed test results for each model in Model Results in the appendix.)

Table 3: Confusion Matrix for Logistic Regression

	Actual 0	Actual 1
Predicted 0	372	164
Predicted 1	50	324

Table 4: Confusion Matrix for Decision Tree Model

	Actual 0	Actual 1
Predicted 0	291	104
Predicted 1	131	384

The logistic regression achieved an overall accuracy of **76.48%**, identifying **66.39%** of spam emails (recall) and **88.15%** of non-spam emails (specificity). This linear model provided a solid baseline but struggled with non-linear patterns. In contrast, the decision tree has an overall accuracy of **74.18%**. It performs better in identifying spam emails with a recall of **78.69%**, but is less effective with non-spam emails with a specificity of **68.96%**.

Since there is clearly trade-off between recall and specificity in this situation, we believe that users will generally prioritize avoiding non-spam emails being incorrectly sent to spam folders, which could lead to missing important emails, over preventing some spam emails slipping into the inbox. Therefore, the logistic regression is a better option without any tuning.

With that being said, we also tried the support vector machine (SVM). We have specifically selected SVM with RBF (Radial Basis Function) kernel due to its ability to handle non-linear patterns effectively. It could potentially outperform logistic regression and

decision tree as our data contains large numbers of outliers. The confusion matrix of testing results is presented in Table 5:

Table 5: Confusion Matrix for Support Vector Machine

	Actual 0	Actual 1
Predicted 0	377	122
Predicted 1	45	366

SVM with RBF delivered an 82% overall accuracy, with 75% recall for spam, and 89%[1] specificity for non-spam. The model minimizes false positives while offering solid spam detection, making it by far the ideal model for our goal of prioritizing non-spam emails retention.

Feature Evaluation

According to our results, the SVM model with RBF kernel yields the best performance with highest overall accuracy, recall, as well as specificity which we belive is the most valuable characteristic in a spam detector. This suggests that the relationships between the target variable and the features are likely non-linear, which may not be fully captured by the correlation heatmap. Therefore, to further assess the importance of each feature, we will compute permutation importance. In this process, each feature is shuffled individually to calculate the model accuracy. The importance of each feature is then determined by the change in accuracy resulting from the shuffling, with feature causing the biggest decline in performance being considered the most important.

[1] The results here are rounded as integers as the results are always slightly different everytime when rendering the pdf, possibly due to cross validation training.

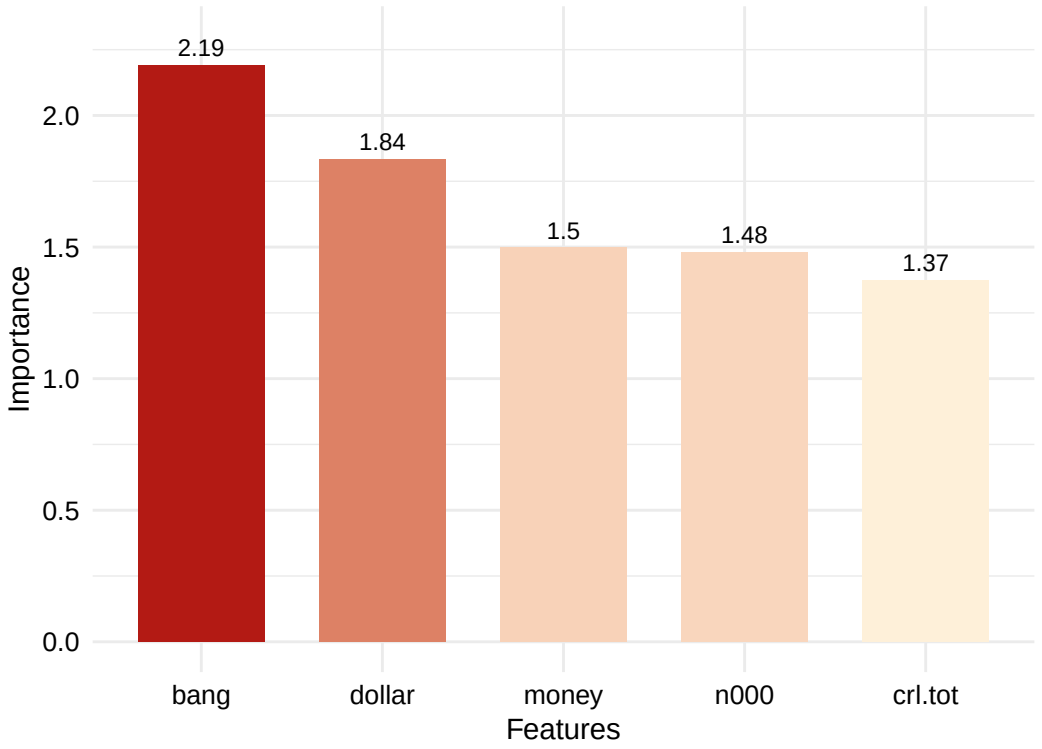

Figure 3: Feature Importance in the SVM(RBF) Model

The permutation importance of each feature is shown above in Figure 3. The results suggest that 'bang' is the most critical feature, with an importance score of 2.19, followed by 'dollar' at 1.84, 'money' at 1.5, 'n000' at 1.48, and 'crl.tot' at 1.37. This contrasts with the correlation heatmap, which showed weak-to-moderate linear correlations between 'yesno' and each feature, with 'n000' and 'dollar' having the highest correlations. The differences are likely due to the SVM's RBF kernel, which is able to capture complex and non-linear relationships. The high importance of 'bang' reflects the pattern of using attention-grabbing marks in spam emails, while 'dollar' and 'money' ranking at second and third indicates that financial terms are also important factors when identifying spam emails. However, it is also worth noticing that the importances of 'n000' and 'crl.tot' are close to money, indicating that they remain significant due to their association with numeric and capital letter patterns in spam emails.

Bibliography

TidyTuesday. (2023). *Spam dataset readme file* Aug. 15, 2023. Retrieved: February 14, 2025, [Online]. Available: https://github.com/rfordatascience/tidytuesday/blob/main/data/2023/2023-08-15/readme.md

H. Wickham, T. Lin Pedersen, and D. Seidel, "Package 'scales,'" Nov. 2023. Accessed: Feb. 23, 2025. [Online]. Available: https://cloud.r-project.org/web/packages/scales/scales.pdf

M. Kuhn *et al.*, "Package 'caret,'" Dec. 2024. Accessed: Feb. 23, 2025. [Online]. Available: https://cran.r-project.org/web/packages/caret/caret.pdf

D. Meyer *et al.*, "Package 'e1071,'" Sep. 2024. Accessed: Feb. 24, 2025. [Online]. Available: https://cran.r-project.org/web/packages/e1071/e1071.pdf

C. Molnar , B. Bischl, and G. Casalicchio, "Package 'iml,'" Journal of Open Source Software, Jun. 2018. Accessed: Feb. 25, 2025. [Online]. Available: https://cran.r-project.org/web/packages/iml/iml.pdf

Appendix

Plots

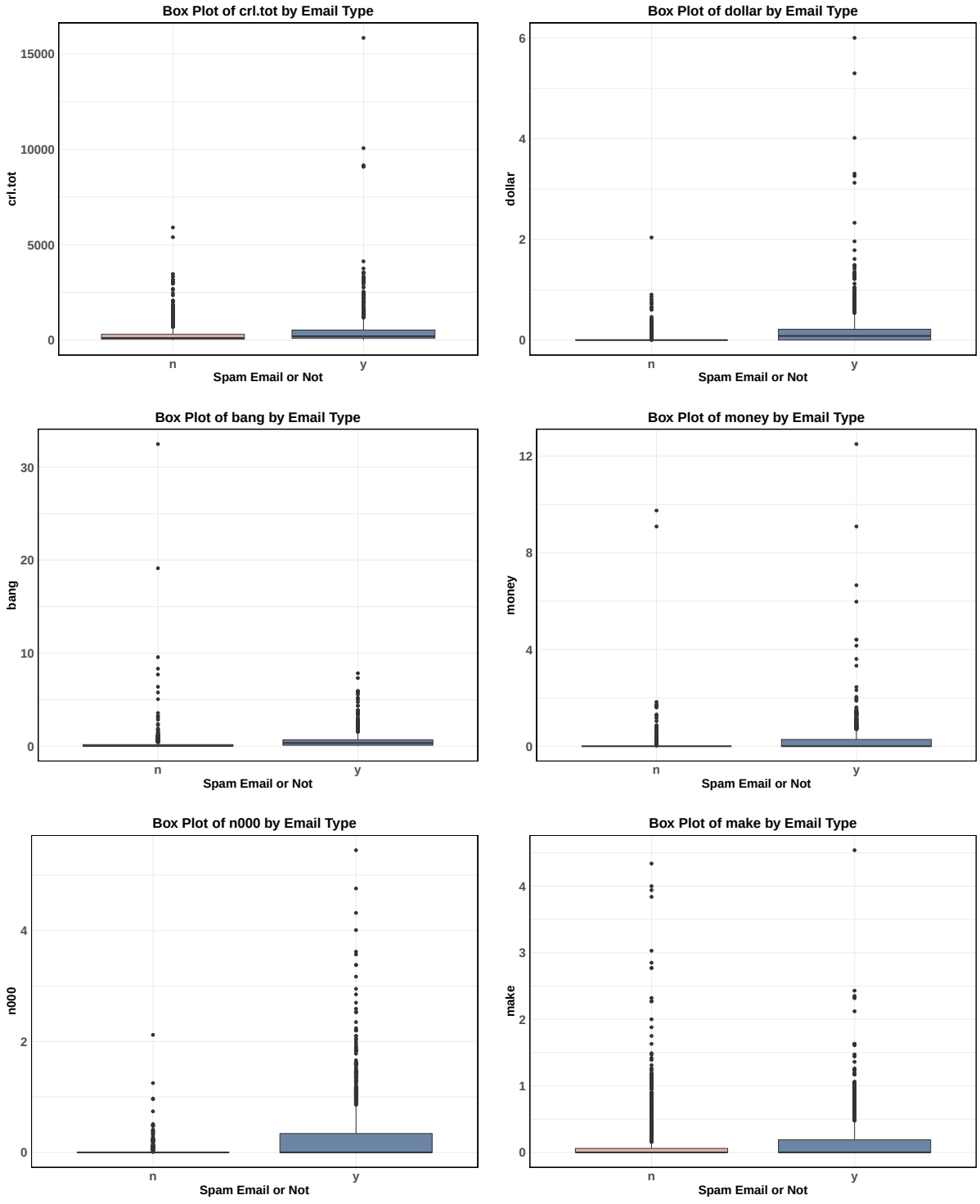

Figure 4: Distribution of each feature in Spam vs Non-spam

Model Results

The model results of logistic regression is as below:

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 372 164
         1  50 324

               Accuracy : 0.7648
                 95% CI : (0.7359, 0.792)
    No Information Rate : 0.5363
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.5357

 Mcnemar's Test P-Value : 1.123e-14

            Sensitivity : 0.6639
            Specificity : 0.8815
         Pos Pred Value : 0.8663
         Neg Pred Value : 0.6940
             Prevalence : 0.5363
         Detection Rate : 0.3560
   Detection Prevalence : 0.4110
      Balanced Accuracy : 0.7727

       'Positive' Class : 1
```

The model results of decision tree is as below:

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 291 104
         1 131 384

               Accuracy : 0.7418
                 95% CI : (0.712, 0.7699)
    No Information Rate : 0.5363
    P-Value [Acc > NIR] : < 2e-16

                  Kappa : 0.4785

 Mcnemar's Test P-Value : 0.08988

            Sensitivity : 0.7869
            Specificity : 0.6896
         Pos Pred Value : 0.7456
         Neg Pred Value : 0.7367
             Prevalence : 0.5363
         Detection Rate : 0.4220
   Detection Prevalence : 0.5659
      Balanced Accuracy : 0.7382

       'Positive' Class : 1
```

The model results of support vector machine (SVM) is as below:

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 377 122
         1  45 366

               Accuracy : 0.8165
                 95% CI : (0.7898, 0.8411)
    No Information Rate : 0.5363
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.6355

 Mcnemar's Test P-Value : 4.077e-09

            Sensitivity : 0.7500
            Specificity : 0.8934
         Pos Pred Value : 0.8905
         Neg Pred Value : 0.7555
             Prevalence : 0.5363
         Detection Rate : 0.4022
   Detection Prevalence : 0.4516
      Balanced Accuracy : 0.8217

       'Positive' Class : 1
```

Code

```
# importing packages

library(readr)
library(ggplot2)
library(knitr)
library(tidyr)
library(dplyr)
library(gridExtra)
library(corrplot)
library(scales)
library(caret)
library(rpart)
library(e1071)
library(kernlab)
library(iml)

# loading dataset
spam_data_raw = readr::read_csv('https://raw.githubusercontent.com/rfordatascience/tid

# Remove duplicates
spam_data = spam_data_raw[!duplicated(spam_data_raw), ]

# compute proportion of yes and no before and after dropping duplicates
proportion_before = spam_data_raw %>%
  count(yesno, name = "count") %>%
  mutate(dataset = "spam_data_raw", proportion = count / sum(count))

proportion_after = spam_data %>%
  count(yesno, name = "count") %>%
  mutate(dataset = "spam_data", proportion = count / sum(count))

# Combine data
yesno_counts = bind_rows(proportion_before, proportion_after)

# plot a stack bar
ggplot(yesno_counts, aes(x = dataset, y = proportion, fill = yesno)) +
  geom_bar(stat = "identity", width = 0.45) +
```

```
  geom_text(aes(label = scales::percent(proportion, accuracy = 0.1)),
            position = position_stack(vjust = 0.5), size = 4, color = "black") +
  scale_fill_manual(values = c("#DEB8AB", "#6C85A4"), name = "Yes/No") +
  labs(x = "Dataset",
       y = "Proportion") +
  theme_minimal() +
  theme(axis.title.y = element_text(size = 11.5),
        axis.title.x = element_text(size = 11.5),
        axis.text.x = element_text(size = 10, color = "black"),
        axis.text.y = element_text(size = 10, color = "black"),
        legend.title = element_text(size = 10),
        legend.text = element_text(size = 10))
# steps here
# 1. define a function to detect and count outliers
# 2. split the data into yes and no
# 3. apply the function to both
# 4. concat the results into a df
# 5. print to see

# include only feature variables
features_columns = colnames(spam_data)[colnames(spam_data) != "yesno"]

# Step 1
detect_outliers = function(x) {
  #define outliers
  IQR_value = IQR(x, na.rm = TRUE)
  Q1 = quantile(x, 0.25, na.rm = TRUE)
  Q3 = quantile(x, 0.75, na.rm = TRUE)
  lower_bound = Q1 - 1.5 * IQR_value
  upper_bound = Q3 + 1.5 * IQR_value
  #detect outliers
  outliers =  x < lower_bound | x > upper_bound
  return(sum(outliers, na.rm = TRUE))
}

# Step 2
spams = spam_data %>% filter(yesno == 'y')
nonspams = spam_data %>% filter(yesno == 'n')

# Step 3
```

```r
spam_outliers = sapply(spams[features_columns], detect_outliers)
nonspam_outliers = sapply(nonspams[features_columns], detect_outliers)

# Step 4
outliers_summary = data.frame(
  #Attribute = numeric_columns,
  Spam_Outliers = spam_outliers,
  Non_Spam_Outliers = nonspam_outliers
)

# Step 5
kable(outliers_summary, caption = "Outliers of Each Attributes in Spam VS Non-Spam Ema
# rescale the data to 0-1
spam_data_normalized = spam_data
spam_data_normalized[features_columns] = lapply(spam_data[features_columns], rescale,

# Encoding y and n to 1 and 0
spam_data_normalized$yesno = ifelse(spam_data_normalized$yesno == "y", 1, 0)
# compute correlation matrix
cor_matrix = cor(spam_data_normalized)

# plot correlation heatmap
corrplot(cor_matrix,
         type = "lower",
         method = "color",
         tl.col = "black",
         tl.srt = 0,
         tl.offset = 0.6,
         tl.cex = 0.6,
         addCoef.col = "black",
         col = colorRampPalette(c("#020060","#FFFFFF","#DEB8AB"))(20),
         cl.cex = 0.6,
         cl.pos = "r",
         number.cex = 0.7
         )
# drop 'make' due to low correlation
spam_final = spam_data_normalized %>% select(-make)

# factoring 1s and 0s
spam_final$yesno = factor(spam_final$yesno, levels = c(0, 1))
```

```r
# split data into training and testing group
set.seed(262626) # group 26 with 3 member...
train_index = createDataPartition(spam_final$yesno, p=0.7, list=FALSE)
train_data = spam_final[train_index, ]
test_data = spam_final[-train_index, ]

# model training
log_reg = train(yesno ~ .,
                data = train_data,
                method = "glm",
                trControl = trainControl(method = "cv", number = 10))

dec_tree = train(yesno ~ .,
                data = train_data,
                method = "rpart",
                trControl = trainControl(method = "cv", number = 10))

# model testing
log_pred = predict(log_reg, test_data)
tree_pred = predict(dec_tree, test_data)

# confusion matrix
log_cm = confusionMatrix(log_pred, test_data$yesno, positive="1")
tree_cm = confusionMatrix(tree_pred, test_data$yesno, positive="1")

# some work for pretty printing
log_cm_df = as.data.frame.matrix(log_cm$table)
colnames(log_cm_df) = paste("Actual", colnames(log_cm_df))
rownames(log_cm_df) = paste("Predicted", rownames(log_cm_df))

tree_cm_df = as.data.frame.matrix(tree_cm$table)
colnames(tree_cm_df) = paste("Actual", colnames(tree_cm_df))
rownames(tree_cm_df) = paste("Predicted", rownames(tree_cm_df))
kable(log_cm_df)
kable(tree_cm_df)
# training SVM
svm = train(yesno ~ .,
                  data = train_data,
                  method = "svmRadial",
                  trControl = trainControl(method = "cv", number = 10),
```

```
                  tuneLength = 10)  # auto tuning...not gonna tune this myself due to

# testing SVM
svm_pred = predict(svm, test_data)

# confusion matrix and pretty printing
svm_cm = confusionMatrix(svm_pred, test_data$yesno, positive="1")
svm_cm_df = as.data.frame.matrix(svm_cm$table)
colnames(svm_cm_df) = paste("Actual", colnames(svm_cm_df))
rownames(svm_cm_df) = paste("Predicted", rownames(svm_cm_df))

# matrix printing
kable(svm_cm_df, caption = "Confusion Matrix for Support Vector Machine")
# computing feature importance
mod = Predictor$new(svm, data = train_data %>% select(-yesno), y = train_data$yesno)
imp = FeatureImp$new(mod, loss = "ce") # set loss = cross-entropy for classification

# Create the bar plot
ggplot(imp$results, aes(x = reorder(feature, -importance), y = importance, fill = impo
  geom_bar(stat = "identity", width = 0.7) +
  scale_fill_gradient(low = "#fef0d9", high = "#b31a14") +
  labs(x = "Features", y = "Importance") +
  theme_minimal() +
  theme(legend.position = "none",
        axis.title.y = element_text(size = 11),
        axis.title.x = element_text(size = 11),
        axis.text.x = element_text(size = 10, color = "black"),
        axis.text.y = element_text(size = 10, color = "black"))+
  geom_text(aes(label = round(importance, 2)), vjust = -0.5, size = 3.2, color = "blac
  ylim(0, 2.3)
#Examining Outliers
plot_boxess = function(df, column) {
  plot = ggplot(df, aes(x = yesno, y = !!sym(column))) +
    geom_boxplot(fill = c("#DEB8AB", "#6C85A4")) +
    labs(title = paste("Box Plot of", column, "by Email Type"),
         x = "Spam Email or Not",
         y = column) +
    theme_minimal() +
    theme(plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
          axis.title.x = element_text(size = 16, face = "bold"),
```

```
          axis.title.y = element_text(size = 16, face = "bold"),
          axis.text.x = element_text(size = 16, face = "bold"),
          axis.text.y = element_text(size = 16, face = "bold"),
          panel.border = element_rect(color = "black", fill = NA, linewidth = 1),
          plot.margin = margin(18, 18, 18, 18))
  return(plot)
}

# include only feature variables
features_columns = colnames(spam_data)[colnames(spam_data) != "yesno"]

# Create an empty list to store the plots
plots = list()

# Generate the plots and store them in the list
for (col in features_columns) {
  plots[[col]] = plot_boxess(spam_data, col)
}

# Arrange the plots into a 3x2 grid with larger plots
grid.arrange(grobs = plots, ncol = 2, nrow = 3)
print(log_cm)
print(tree_cm)
print(svm_cm)
```